

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Автоэнкодеры»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа

Ход работы

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

9	Wine Quality (white)	quality
---	--------------------------------------	---------

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE, trustworthiness
import sklearn
import warnings
warnings.filterwarnings('ignore')

print("scikit-learn version:", sklearn.__version__)

print("=" * 60)
print("1. DATA LOADING")
print("=" * 60)

url_white = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv"
white_wine = pd.read_csv(url_white, sep=';')

print(f"Dataset shape: {white_wine.shape}")
print(f"Columns: {list(white_wine.columns)}")
print(f"\nQuality distribution:")
print(white_wine['quality'].value_counts().sort_index())

X = white_wine.drop(columns=['quality']).values
y = white_wine['quality'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

def group_quality(q):
    if q <= 4:
        return 0
    elif q == 5:
```

```

        return 1
    elif q == 6:
        return 2
    else:
        return 3

y_grouped = np.array([group_quality(q) for q in y])

print(f"\nGrouped class distribution:")
unique, counts = np.unique(y_grouped, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  Class {u}: {c} samples")

class_names = {0: 'Low (3-4)', 1: 'Medium (5)', 2: 'High (6)', 3: 'Very High (7-9)'}
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12']
markers = ['o', 's', '^', 'D']

print("\n" + "=" * 60)
print("2. AUTOENCODER ARCHITECTURE SEARCH")
print("=" * 60)

X_tensor = torch.FloatTensor(X_scaled)
dataset = TensorDataset(X_tensor, X_tensor)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)
input_dim = X_scaled.shape[1]

class Autoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim, hidden_dim=32):
        super(Autoencoder, self).__init__()

        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim),
            nn.ReLU()
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim)
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

    def encode(self, x):
        return self.encoder(x)

def train_autoencoder(model, dataloader, epochs=100, lr=1e-3):
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    losses = []
    model.train()

    for epoch in range(epochs):
        epoch_loss = 0.0
        for batch_x, batch_y in dataloader:
            optimizer.zero_grad()
            output = model(batch_x)
            loss = criterion(output, batch_y)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item() * batch_x.size(0)

        epoch_loss /= len(dataloader.dataset)
        losses.append(epoch_loss)

    if (epoch + 1) % 20 == 0:
        print(f"  Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss:.6f}")

    return losses

```

```

def get_latent_representation(model, X_tensor):
    model.eval()
    with torch.no_grad():
        latent = model.encode(X_tensor).numpy()
    return latent

configs = [
    {"hidden_dim": 16, "lr": 1e-3, "epochs": 100},
    {"hidden_dim": 32, "lr": 1e-3, "epochs": 100},
    {"hidden_dim": 64, "lr": 1e-3, "epochs": 100},
]

print("\n--- Architecture search for latent_dim = 2 ---")
search_results_2d = {}
for cfg in configs:
    print(f"\n Config: hidden_dim={cfg['hidden_dim']}, lr={cfg['lr']}, epochs={cfg['epochs']}")
    ae = Autoencoder(input_dim=input_dim, latent_dim=2, hidden_dim=cfg["hidden_dim"])
    losses = train_autoencoder(ae, dataloader, epochs=cfg["epochs"], lr=cfg["lr"])
    search_results_2d[cfg["hidden_dim"]] = losses[-1]
    print(f" Final loss: {losses[-1]:.6f}")

print("\n--- Architecture search for latent_dim = 3 ---")
search_results_3d = {}
for cfg in configs:
    print(f"\n Config: hidden_dim={cfg['hidden_dim']}, lr={cfg['lr']}, epochs={cfg['epochs']}")
    ae = Autoencoder(input_dim=input_dim, latent_dim=3, hidden_dim=cfg["hidden_dim"])
    losses = train_autoencoder(ae, dataloader, epochs=cfg["epochs"], lr=cfg["lr"])
    search_results_3d[cfg["hidden_dim"]] = losses[-1]
    print(f" Final loss: {losses[-1]:.6f}")

best_hidden_2d = min(search_results_2d, key=search_results_2d.get)
best_hidden_3d = min(search_results_3d, key=search_results_3d.get)

print(f"\nBest hidden_dim for latent_dim=2: {best_hidden_2d} (loss={search_results_2d[best_hidden_2d]:.6f})")
print(f"Best hidden_dim for latent_dim=3: {best_hidden_3d} (loss={search_results_3d[best_hidden_3d]:.6f})")

print("\n--- Training final autoencoder with 2 latent neurons ---")
ae_2d = Autoencoder(input_dim=input_dim, latent_dim=2, hidden_dim=best_hidden_2d)
losses_2d = train_autoencoder(ae_2d, dataloader, epochs=100, lr=1e-3)
latent_2d = get_latent_representation(ae_2d, X_tensor)

print("\n--- Training final autoencoder with 3 latent neurons ---")
ae_3d = Autoencoder(input_dim=input_dim, latent_dim=3, hidden_dim=best_hidden_3d)
losses_3d = train_autoencoder(ae_3d, dataloader, epochs=100, lr=1e-3)
latent_3d = get_latent_representation(ae_3d, X_tensor)

print("\nAutoencoders trained.")

print("\n" + "=" * 60)
print("3. PCA")
print("=" * 60)

pca_2d = PCA(n_components=2, random_state=42)
pca_2d_result = pca_2d.fit_transform(X_scaled)
print(f"PCA 2D: explained variance = {pca_2d.explained_variance_ratio_.sum():.4f}")

pca_3d = PCA(n_components=3, random_state=42)
pca_3d_result = pca_3d.fit_transform(X_scaled)
print(f"PCA 3D: explained variance = {pca_3d.explained_variance_ratio_.sum():.4f}")

print("\n" + "=" * 60)
print("4. t-SNE")
print("=" * 60)

perplexity_values = [20, 30, 40, 50, 60]
tsne_2d_results = {}
tsne_3d_results = {}

np.random.seed(42)
sample_indices = np.random.choice(len(X_scaled), size=min(1000, len(X_scaled)), replace=False)
X_sample = X_scaled[sample_indices]
y_sample = y_grouped[sample_indices]

print(f"Subsample used: {len(sample_indices)} samples")

for perp in perplexity_values:
    print(f"\n Perplexity = {perp}")

```

```

tsne_2d = TSNE(
    n_components=2,
    perplexity=perp,
    init='pca',
    random_state=42,
    learning_rate='auto',
    max_iter=1000
)
tsne_2d_result = tsne_2d.fit_transform(X_sample)
tsne_2d_results[perp] = tsne_2d_result
print(f"    2D done")

tsne_3d = TSNE(
    n_components=3,
    perplexity=perp,
    init='pca',
    random_state=42,
    learning_rate='auto',
    max_iter=1000
)
tsne_3d_result = tsne_3d.fit_transform(X_sample)
tsne_3d_results[perp] = tsne_3d_result
print(f"    3D done")

print("\nt-SNE completed.")

print("\n" + "=" * 60)
print("5. SELECTING BEST PERPLEXITY BY TRUSTWORTHINESS")
print("=" * 60)

trust_scores = {}
for perp in perplexity_values:
    score = trustworthiness(X_sample, tsne_2d_results[perp], n_neighbors=5)
    trust_scores[perp] = score
    print(f"    Perplexity = {perp}: trustworthiness = {score:.4f}")

best_perp = max(trust_scores, key=trust_scores.get)
print(f"\nBest perplexity: {best_perp} (trustworthiness = {trust_scores[best_perp]:.4f})")

print("\n" + "=" * 60)
print("6. VISUALIZATION")
print("=" * 60)

fig, ax = plt.subplots(figsize=(10, 8))
for cls in np.unique(y_grouped):
    mask = y_grouped == cls
    ax.scatter(
        latent_2d[mask, 0], latent_2d[mask, 1],
        c=colors[cls], marker=markers[cls], label=class_names[cls],
        alpha=0.6, s=20, edgecolors='w', linewidth=0.3
    )
ax.set_title('Autoencoder (2D latent) - Wine Quality (white)', fontsize=14, fontweight='bold')
ax.set_xlabel('Latent Dimension 1')
ax.set_ylabel('Latent Dimension 2')
ax.legend(title='Quality Class', loc='best')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ae_2d.png', dpi=150, bbox_inches='tight')
plt.show()

fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')
for cls in np.unique(y_grouped):
    mask = y_grouped == cls
    ax.scatter(
        latent_3d[mask, 0], latent_3d[mask, 1], latent_3d[mask, 2],
        c=colors[cls], marker=markers[cls], label=class_names[cls],
        alpha=0.6, s=20, edgecolors='w', linewidth=0.3
    )
ax.set_title('Autoencoder (3D latent) - Wine Quality (white)', fontsize=14, fontweight='bold')
ax.set_xlabel('Latent 1')
ax.set_ylabel('Latent 2')
ax.set_zlabel('Latent 3')
ax.legend(title='Quality Class', loc='best')
plt.tight_layout()
plt.savefig('ae_3d.png', dpi=150, bbox_inches='tight')
plt.show()

```

```

fig, ax = plt.subplots(figsize=(10, 8))
for cls in np.unique(y_grouped):
    mask = y_grouped == cls
    ax.scatter(
        pca_2d_result[mask, 0], pca_2d_result[mask, 1],
        c=colors[cls], marker=markers[cls], label=class_names[cls],
        alpha=0.6, s=20, edgecolors='w', linewidth=0.3
    )
ax.set_title('PCA (2 components) - Wine Quality (white)', fontsize=14, fontweight='bold')
ax.set_xlabel(f'PC1 ({pca_2d.explained_variance_ratio_[0]:.2%})')
ax.set_ylabel(f'PC2 ({pca_2d.explained_variance_ratio_[1]:.2%})')
ax.legend(title='Quality Class', loc='best')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('pca_2d.png', dpi=150, bbox_inches='tight')
plt.show()

fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')
for cls in np.unique(y_grouped):
    mask = y_grouped == cls
    ax.scatter(
        pca_3d_result[mask, 0], pca_3d_result[mask, 1], pca_3d_result[mask, 2],
        c=colors[cls], marker=markers[cls], label=class_names[cls],
        alpha=0.6, s=20, edgecolors='w', linewidth=0.3
    )
ax.set_title('PCA (3 components) - Wine Quality (white)', fontsize=14, fontweight='bold')
ax.set_xlabel(f'PC1 ({pca_3d.explained_variance_ratio_[0]:.2%})')
ax.set_ylabel(f'PC2 ({pca_3d.explained_variance_ratio_[1]:.2%})')
ax.set_zlabel(f'PC3 ({pca_3d.explained_variance_ratio_[2]:.2%})')
ax.legend(title='Quality Class', loc='best')
plt.tight_layout()
plt.savefig('pca_3d.png', dpi=150, bbox_inches='tight')
plt.show()

fig, axes = plt.subplots(1, len(perplexity_values), figsize=(4 * len(perplexity_values), 4))
for idx, perp in enumerate(perplexity_values):
    ax = axes[idx]
    result = tsne_2d_results[perp]
    for cls in np.unique(y_sample):
        mask = y_sample == cls
        ax.scatter(
            result[mask, 0], result[mask, 1],
            c=colors[cls], marker=markers[cls],
            alpha=0.6, s=15, edgecolors='w', linewidth=0.2
        )
    ax.set_title(f'Perplexity = {perp}', fontsize=11)
    ax.set_xticks([])
    ax.set_yticks([])
fig.suptitle('t-SNE (2D) - Wine Quality (white), various perplexity', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('tsne_2d_perplexity.png', dpi=150, bbox_inches='tight')
plt.show()

result_3d = tsne_3d_results[best_perp]

fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')
for cls in np.unique(y_sample):
    mask = y_sample == cls
    ax.scatter(
        result_3d[mask, 0], result_3d[mask, 1], result_3d[mask, 2],
        c=colors[cls], marker=markers[cls], label=class_names[cls],
        alpha=0.6, s=20, edgecolors='w', linewidth=0.3
    )
ax.set_title(f't-SNE (3D, perplexity={best_perp}) - Wine Quality (white)', fontsize=14, fontweight='bold')
ax.set_xlabel('t-SNE 1')
ax.set_ylabel('t-SNE 2')
ax.set_zlabel('t-SNE 3')
ax.legend(title='Quality Class', loc='best')
plt.tight_layout()
plt.savefig('tsne_3d.png', dpi=150, bbox_inches='tight')
plt.show()

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

ax = axes[0]
for cls in np.unique(y_grouped):
    mask = y_grouped == cls

```

```

        ax.scatter(latent_2d[mask, 0], latent_2d[mask, 1],
                    c=colors[cls], marker=markers[cls], alpha=0.5, s=15)
ax.set_title('Autoencoder (2D)', fontsize=12, fontweight='bold')
ax.set_xlabel('Latent 1')
ax.set_ylabel('Latent 2')
ax.grid(True, alpha=0.3)

ax = axes[1]
for cls in np.unique(y_grouped):
    mask = y_grouped == cls
    ax.scatter(pca_2d_result[mask, 0], pca_2d_result[mask, 1],
                c=colors[cls], marker=markers[cls], alpha=0.5, s=15)
ax.set_title('PCA (2D)', fontsize=12, fontweight='bold')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.grid(True, alpha=0.3)

ax = axes[2]
result = tsne_2d_results[best_perp]
for cls in np.unique(y_sample):
    mask = y_sample == cls
    ax.scatter(result[mask, 0], result[mask, 1],
                c=colors[cls], marker=markers[cls], alpha=0.5, s=15)
ax.set_title(f't-SNE (2D, perplexity={best_perp})', fontsize=12, fontweight='bold')
ax.set_xlabel('t-SNE 1')
ax.set_ylabel('t-SNE 2')
ax.grid(True, alpha=0.3)

handles = [plt.Line2D([0], [0], marker=markers[i], color='w',
                      markerfacecolor=colors[i], markersize=8, label=class_names[i])
            for i in range(4)]
fig.legend(handles=handles, title='Quality Class', loc='lower center',
           ncol=4, bbox_to_anchor=(0.5, -0.05))

fig.suptitle('Comparison of dimensionality reduction methods - Wine Quality (white)',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('comparison_2d.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nAll visualizations saved to the current directory.")
print("Done!")

```

Результат программы:

scikit-learn version: 1.9.1

1. DATA LOADING

Dataset shape: (4898, 12)

Columns: ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol', 'quality']

Quality distribution:

quality	count
3	20
4	163
5	1457
6	2198
7	880
8	175
9	5

Name: count, dtype: int64

Grouped class distribution:

Class 0: 183 samples

Class 1: 1457 samples
Class 2: 2198 samples
Class 3: 1060 samples

2. AUTOENCODER ARCHITECTURE SEARCH

--- Architecture search for latent_dim = 2 ---

Config: hidden_dim=16, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.490001
Epoch [40/100], Loss: 0.473278
Epoch [60/100], Loss: 0.463694
Epoch [80/100], Loss: 0.456052
Epoch [100/100], Loss: 0.450674
Final loss: 0.450674

Config: hidden_dim=32, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.521303
Epoch [40/100], Loss: 0.478316
Epoch [60/100], Loss: 0.445915
Epoch [80/100], Loss: 0.433679
Epoch [100/100], Loss: 0.425341
Final loss: 0.425341

Config: hidden_dim=64, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.477070
Epoch [40/100], Loss: 0.443146
Epoch [60/100], Loss: 0.428197
Epoch [80/100], Loss: 0.418964
Epoch [100/100], Loss: 0.413343
Final loss: 0.413343

--- Architecture search for latent_dim = 3 ---

Config: hidden_dim=16, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.419008
Epoch [40/100], Loss: 0.384918
Epoch [60/100], Loss: 0.370038
Epoch [80/100], Loss: 0.361461
Epoch [100/100], Loss: 0.354325
Final loss: 0.354325

Config: hidden_dim=32, lr=0.001, epochs=100
Epoch [20/100], Loss: 0.373050
Epoch [40/100], Loss: 0.342943
Epoch [60/100], Loss: 0.327892
Epoch [80/100], Loss: 0.320382
Epoch [100/100], Loss: 0.314900

Final loss: 0.314900

Config: hidden_dim=64, lr=0.001, epochs=100

Epoch [20/100], Loss: 0.344186

Epoch [40/100], Loss: 0.321674

Epoch [60/100], Loss: 0.310300

Epoch [80/100], Loss: 0.302499

Epoch [100/100], Loss: 0.296252

Final loss: 0.296252

Best hidden_dim for latent_dim=2: 64 (loss=0.413343)

Best hidden_dim for latent_dim=3: 64 (loss=0.296252)

--- Training final autoencoder with 2 latent neurons ---

Epoch [20/100], Loss: 0.507365

Epoch [40/100], Loss: 0.446942

Epoch [60/100], Loss: 0.430965

Epoch [80/100], Loss: 0.422226

Epoch [100/100], Loss: 0.415992

--- Training final autoencoder with 3 latent neurons ---

Epoch [20/100], Loss: 0.336322

Epoch [40/100], Loss: 0.314169

Epoch [60/100], Loss: 0.305212

Epoch [80/100], Loss: 0.298186

Epoch [100/100], Loss: 0.291925

Autoencoders trained.

3. PCA

PCA 2D: explained variance = 0.4361

PCA 3D: explained variance = 0.5472

4. t-SNE

Subsample used: 1000 samples

Perplexity = 20

2D done

3D done

Perplexity = 30

2D done

3D done

Perplexity = 40

2D done

3D done

Perplexity = 50

2D done

3D done

Perplexity = 60

2D done

3D done

t-SNE completed.

5. SELECTING BEST PERPLEXITY BY TRUSTWORTHINESS

Perplexity = 20: trustworthiness = 0.9804

Perplexity = 30: trustworthiness = 0.9779

Perplexity = 40: trustworthiness = 0.9778

Perplexity = 50: trustworthiness = 0.9770

Perplexity = 60: trustworthiness = 0.9741

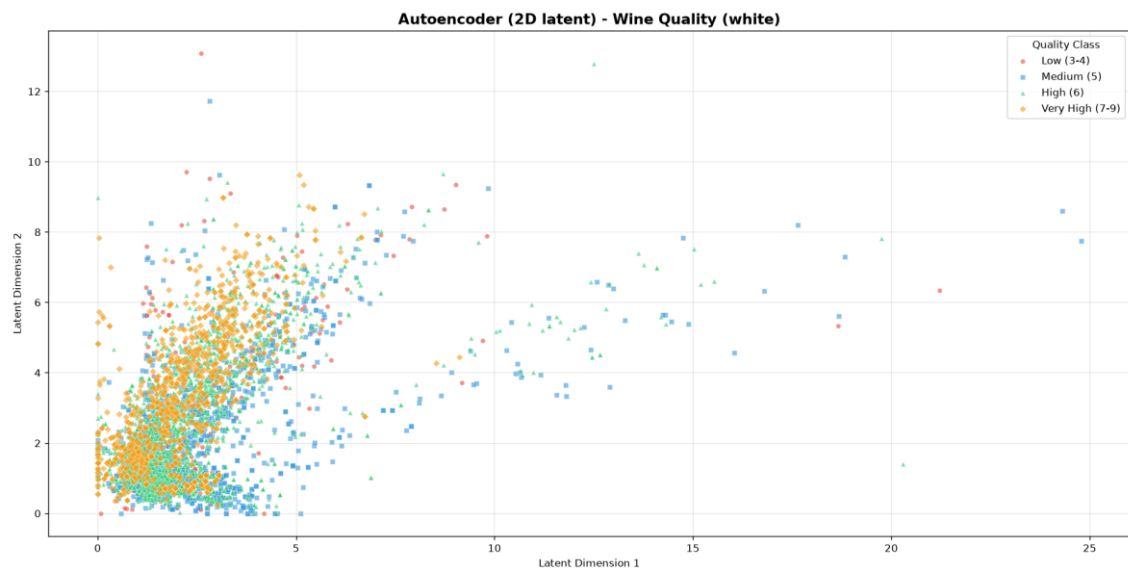
Best perplexity: 20 (trustworthiness = 0.9804)

6. VISUALIZATION

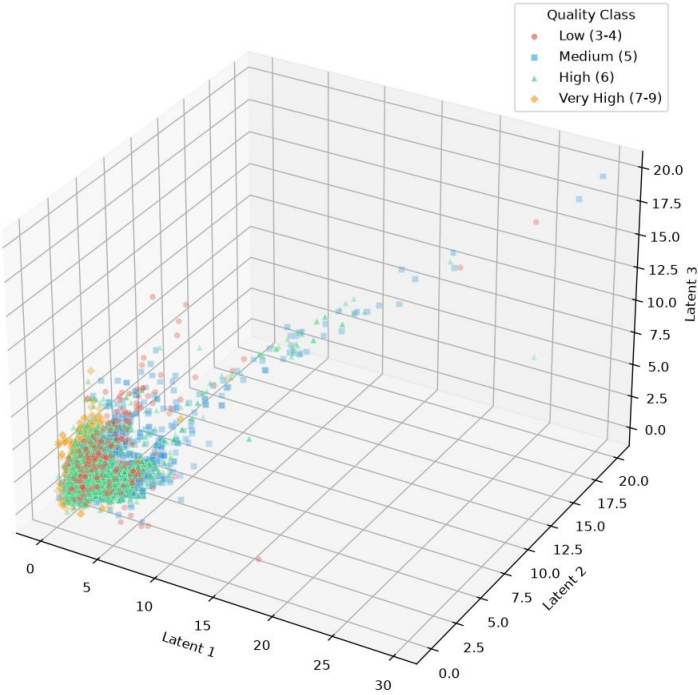
All visualizations saved to the current directory.

Done!

Визуализация:



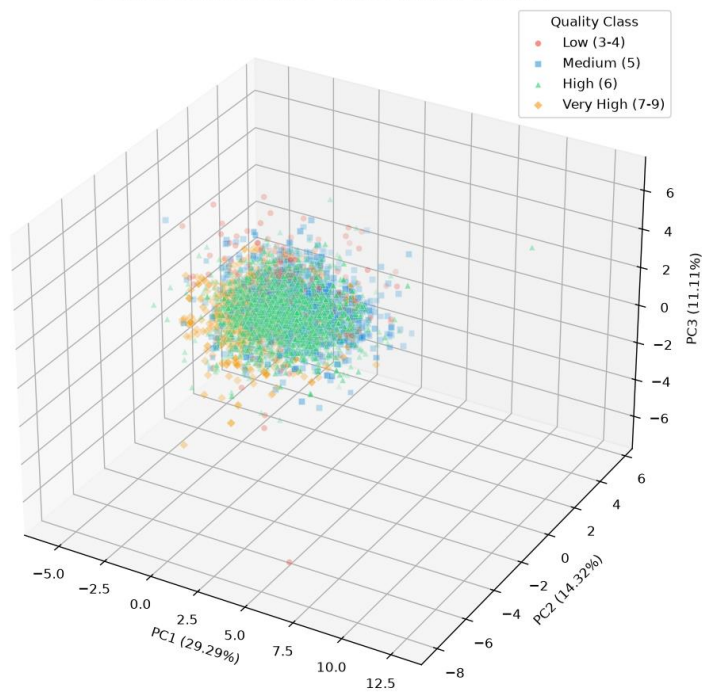
Autoencoder (3D latent) - Wine Quality (white)



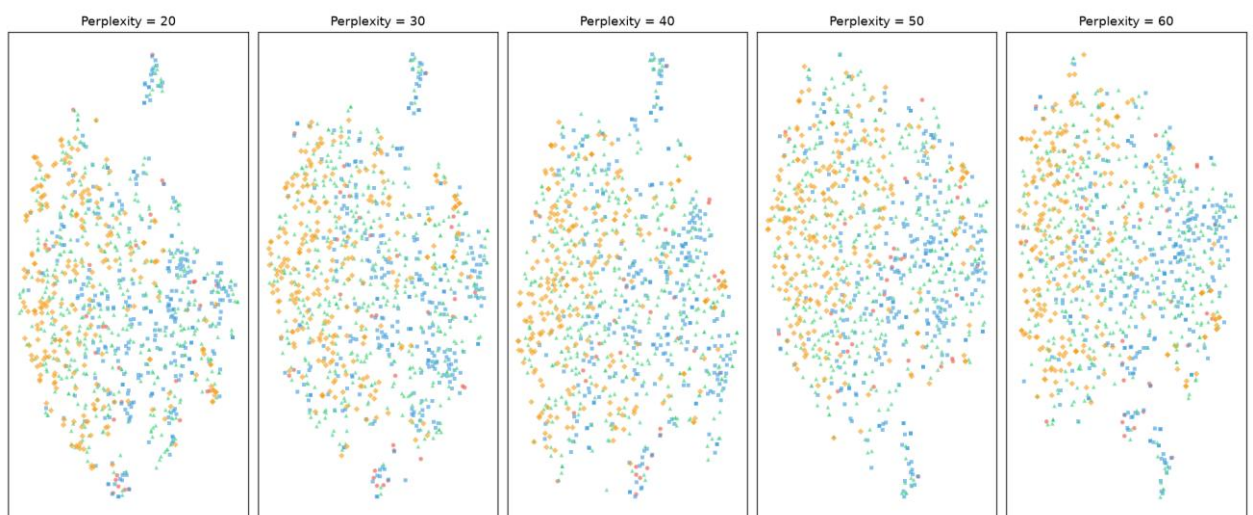
PCA (2 components) - Wine Quality (white)



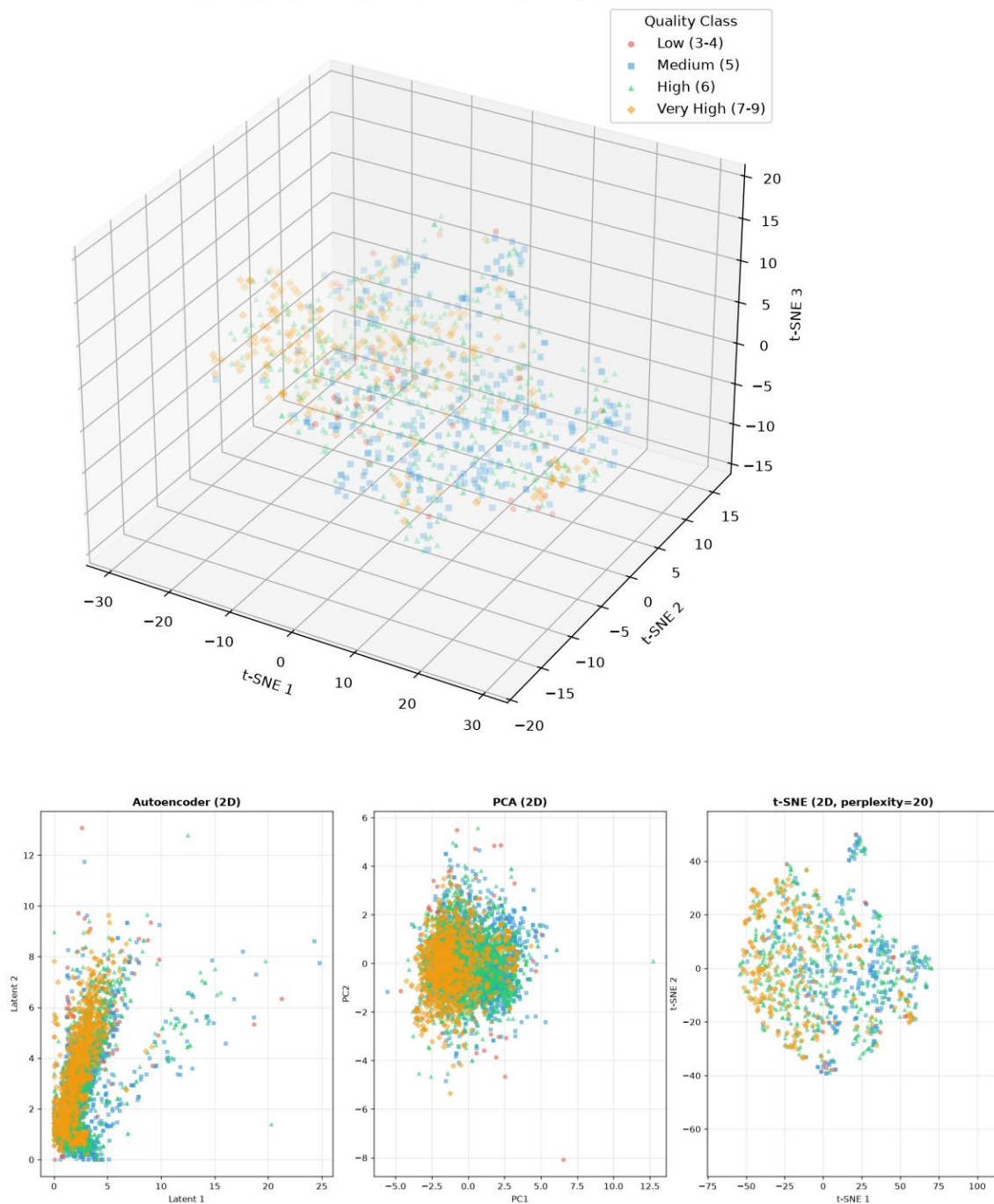
PCA (3 components) - Wine Quality (white)



t-SNE (2D) - Wine Quality (white), various perplexity



t-SNE (3D, perplexity=20) - Wine Quality (white)



В работе выполнена визуализация данных Wine Quality (white): 4898 образцов, 11 признаков, целевая переменная quality от 3 до 9 с сильным дисбалансом классов (доминирует оценка 6). Для визуализации оценки сгруппированы в четыре класса: Low (3–4), Medium (5), High (6), Very High (7–9), при этом Low оказался крайне малочисленным и практически неразличим на графиках. Автоэнкодер обучался экспериментально с подбором архитектуры; лучший результат дала конфигурация со скрытым слоем из 32 нейронов. Ошибка восстановления составила около 0.436 для двух латентных нейронов и 0.321 для трёх, что подтверждает преимущество трёхмерного представления. Визуально автоэнкодер даёт умеренное разделение классов.

PCA объясняет лишь 43.61% дисперсии в 2D и 54.72% в 3D, классы полностью перемешаны: линейный метод не справляется с нелинейной структурой данных. t-SNE показал наилучшую разделимость: при малых значениях perplexивности (20–30) формируются выраженные локальные кластеры, особенно для классов Medium и Very High.

Итог: для данной задачи нелинейные методы предпочтительнее PCA. t-SNE лучше подходит для исследовательского анализа, автоэнкодер: для построения прогнозных моделей. Главная проблема визуализации: дисбаланс классов.

Вывод: научилась применять автоэнкодеры для осуществления визуализации данных и их анализа.